Introduction to Computers and Python

Adapted from “Intro to Python” By Paul Deitel and Harvey Deitel

Introduction

Python is powerful and popular

It enables you to make computers do what you want

Software controls the hardware

Historically programs ran on a single computer

Hardware and software

Computers make calculations faster than a human

Supercomputers do so to an even greater degree

Computers work under a set of instructions

Hardware includes:

- CPU
- RAM
- SSD
- Keyboard/mice

Hardware and software

Many things increase in price over time

Computers largely don’t

Much like price, size tends to decrease

Based on silicon-chip technology

Moore’s law

Named for Gordon Moore

For several decades hardware costs fell rapidly

About every year or two the capacities of computers has approximately doubled

- This trend is called **Moore's Law**

In terms of computers relates primarily to:

- Memory
- Secondary storage
- Processor speeds

Computer Organization

Regardless of appearance computers can be broken up into a set of logical units

- Input Unit
- Output Unit
- Memory unit
- Arithmetic Unit (ALU)
- Central Processing Unit (CPU)
- Secondary Storage Unit

Input unit

The “receiving” section of a computer

Often user input is from:

- Keyboards
- Mice
- Touchscreens

Output unit

The “shipping” section

Often information sent to the screen

But there are several other possibilities

Some directly for viewing

Others not

Memory unit

Rapid access “warehouse”

Retains information

Makes it immediately available

Information is volatile

Memory, primary memory, or RAM

Arithmetic and Logic Unit (ALU)

The “manufacturing” section

Performs calculations

Checks equality

Part of the CPU

Central Processing Unit

The “administrative” section

Coordinates and supervises the operation of other sections

Most computers have multi-core systems

Secondary Storage Unit

Long-term high capacity

Data held until needed

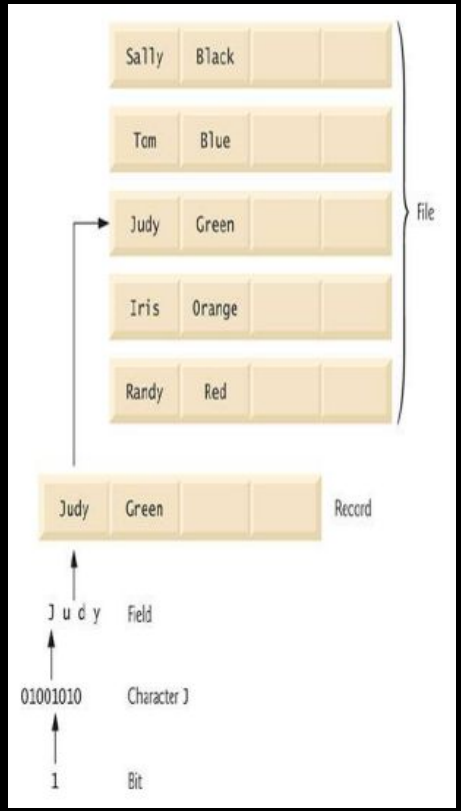
Persistent

Longer access times

Data Hierarchy

Data processed forms a hierarchy

Becomes larger and more complex as it goes



Bits

Short for binary digit

A zero or one (0.1)

Smallest form of data

Basis for binary

Characters

No one wants to work only in bits

Decimal numbers preferred

Or letters

Computer's character set includes every character as a pattern of 1s and 0s

Python uses unicode

The ascii subset can be found at http://unicode.org/charts/PDF/U0000.pdf

The unicode charts for all languages, symbols, emoji, are viewable at http://www.unicode.org/charts/

Fields

Composed of characters, or bytes

A field conveys a meaning

Could represent a name

Or an age

Records

Built with related fields

Could be something like a payroll system

Representing an individual employee

Files

A group of related records

Arbitrary data

Often a sequence of bytes

Often people have many files

Databases

A collection of data

Easy access and manipulation

Relational databases are the most popular

Big data

Tons of data produced globally

The rate is accelerating

Unit	Bytes	Which is approximately
1 kilobyte (KB)	1024 bytes	10^3 (1024) bytes exactly
1 megabyte (MB)	1024 kilobytes	10^6 (1,000,000) bytes
1 gigabyte (GB)	1024 megabytes	10^9 (1,000,000,000) bytes
1 terabyte (TB)	1024 gigabytes	10^{12} (1,000,000,000,000) bytes
1 petabyte (PB)	1024 terabytes	10^{15} (1,000,000,000,000,000) bytes
1 exabyte (EB)	1024 petabytes	10^{18} (1,000,000,000,000,000,000) bytes
1 zettabyte (ZB)	1024 exabytes	10^{21} (1,000,000,000,000,000,000,000) bytes

1.3-2 Full Alternative Text

Python

The focus of this course

Released in 1991

One of the most popular languages

Problem Solving

Based on “Thinking Like a Programmer” By V. Anton Spraul

Strategies for Problem Solving

Problems

Contain constraints

Programming has constraints as well

Programming can define solving a problem as writing a unique program to perform a particular set of tasks under the constraints

Constraints

Take for example a program to print out all the students is the program correct if:

- It only prints the first x number of students
- It skips students with an s in their name
- It prints students in a random order

The Fox, the Goose, and the Corn

“A farmer with a fox, a goose, and a sack of corn need to cross a river. The farmer has a rowboat, but there is only room for the farmer and one of his three items. Unfortunately, both the fox and the goose are hungry. The fox cannot be left alone with the goose, or the fox will eat the goose. Likewise, the goose cannot be left alone with the sack of corn, or the goose will eat the corn. How does the farmer get everything across the river?

The Fox, the Goose, and the Corn- Constraints

Problem- Get all three across without losing something

Fox and Goose can’t be alone

Goose and Corn can’t be alone

Only one at a time

How do we start?

The Fox, the Goose, and the Corn

The fox can’t be left with the goose

Goose can’t be left with the corn

So we know first the goose must be taken across the river

Now the fox and corn are on the first shore

The goose on the second

Now What Does He Take?

Taking either the fox or the corn to the goose will cause an issue

The Fox, the Goose, and the Corn - Think Outside the Box

The goose is on the far shore, does it need to stay there?

The Fox, the Goose, and the Corn - Think Outside the Box

The goose is on the far shore, does it need to stay there?

Nope!

The Fox, the Goose, and the Corn - Think Outside the Box

After leaving the goose on the second shore the farmer returns to the first shore

Here the farmer picks up the fox

Takes it to the second shore

At the second shore he switches out the fox and the goose

The Fox, the Goose, and the Corn

See where this is going?

The corn is on the first shore

The fox on the second shore

And the goose in the boat with the farmer

Do you see the solution?

Outline the remaining steps to yourself now

The Fox, the Goose, and the Corn

The farmer goes back with the goose and switches it with the corn

The farmer returns to the second shore leaving the corn with the fox

Finally the farmer collects the goose from the first shore and returns to the second with all his belongings intact

The Fox, the Goose, and the Corn

Did you figure this out yourself? If so at what stage?

If you didn’t that’s ok

However this riddle is a lot more manageable when you break out the parts

Breaking it out showed who to take first, and from there you can work out the following steps

That’s programming, you don’t always know how to do something but if you break it into parts and find where to start it’s easier to figure out how to get there

The hard part is inferring that items can travel between shores

General Steps to Solve a Problem

Solving a Problem: Identify The Problem

This may seem like a gimme

You can’t solve a problem if you aren’t clear on what you’re trying to solve

Solving a Problem: Always Have a Plan

Having a plan is paramount, and always doable

Plans can be altered as the are implemented, they don’t have to be set in stone

Plans allow intermediate goals to be set, and plans to be reevaluated at those stages

Intermediate goals allow for smaller steps and less frustration along the way

Solving a Problem: Restate the problem

Restating problems can produce valuable results

Looking at problems in different lights can make ones that formally seemed hard seem easy

Imagine you have to climb a hill, why not circle it first to spot the easiest way up?

Restatement also helps with understanding

Solving a Problem: Divide the Problem

Breaking the problem apart can make it much easier to handle

The individual pieces may be easier to solve than the problem as a whole

Solving a Problem: Start with What You Know

Start with what you know how to do, and work out from there

Having a partial solution can make it more clear how to finish the whole problem

Solving a Problem: Reduce the Problem

When faced with a problem you can’t solve try adding or removing constraints

Suppose you have to find the coordinates in 3d space that are closest together. Do you know how to solve this? What if they are in 2d space, can that be figured out and expanded upon that solution?

Reductions are used to simplify the problem so the simplified version can be solved, then the solution can be expanded for the whole problem

Solving a Problem: Look for Analogies

Analogies can be described as similar problems

If you figure out how to solve the simpler/similar problem, you can branch over to the actual problem

A lot of analogies aren’t direct comparisons, so some insight is required

Recognizing analogies is an important skill

Solving a Problem: Experiment

Don’t be afraid to experiment

You can play with different aspects to see how changes alter the outcome

Experiment in a controlled fashion, so you are able to correlate the changes you make to the changes in the outcome

Start small with your experimentation

Solving a Problem: Don’t get Frustrated

When you’re frustrated you don’t think as clearly

Take a break and come back with fresh eyes

Just because it isn’t working now, doesn’t mean it won’t ever

Frustrated programmers are frustrated at themselves, so let yourself off the hook and take a break!